UNIT IV

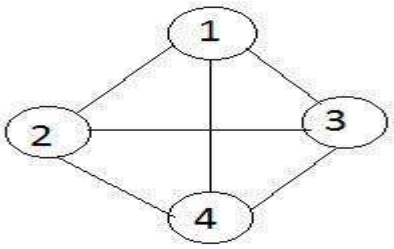
Graph: Terminology, Representation, Traversals – Applications - spanning trees, shortest path and Transitive closure, Topological sort. Sets: Representation - Operations on sets – Applications

2 MARKS

1. Define graph

- A graph consists of two sets V, E.
- V is a finite and non empty set of vertices.
- E is a set of pair of vertices; each pair is called an edge.
- V(G),E(G) represents set of vertices ,set of edges .

G= (V,E)



2. Define digraph (Nov 13)

If an edge between any two nodes in a graph is directionally oriented, a graph is called as directed .it is also referred as digraph.

3. Define undirected graph.

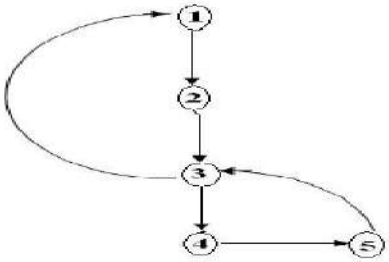
If an edge between any two nodes in a graph is not directionally oriented, a graph is called as undirected .it is also referred as unqualified graph.

4. Define path in a graph

A path in a graph is defined as a sequence of distinct vertices each adjacent to the next except possibly the first vertex and last vertex is different.

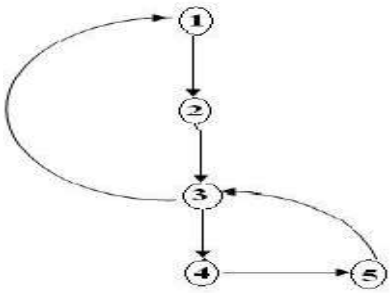
5. Define a cycle in a graph

A cycle is a path containing atleast three vertices such that the starting and the ending vertices are the same. The cycles are (1, 2, 3, 1), (1, 2, 3, 4, 5, 3, 1)



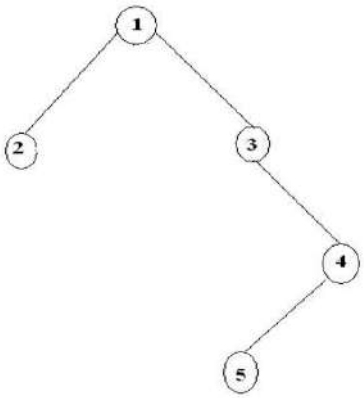
6. Define a strongly connected graph (Nov 14)

A graph is said to be a strongly connected graph, if for every pair of distinct vertices there is a directed path from every vertex to every other vertex. It is also referred as a complete graph.



7. Define a weakly connected graph. (May 14)

A directed graph is said to be a weakly connected graph if any vertex doesn't have a directed path to any other vertices.



8. Define a weighted graph.

A graph is said to be a weighted graph if every edge in the graph is assigned some weight or value. The weight of an edge is a positive value that may be representing the distance between the vertices or the weights of the edges along the path.

9. How we can represent the graph?

We can represent the graph by three ways

1.adjacent matrix

2.adjacent list

3. adjacent multi list

10. Define adjacency matrix

Adjacency matrix is a representation used to represent a graph with zeros and ones.

A graph containing n vertices can be represented by a matrix with n rows and n columns.

The matrix is formed by storing 1 n its i^{th} and j^{th} column of the matrix, if there exists an edge between ith and jth vertex of the graph and 0 if there is no edge between i^{th} and j^{th} vertex of the graph .
Adjacency matrix is also referred as incidence matrix.

11. What is meant by traversing a graph? State the different ways of traversing a graph. (Nov 12)

In undirected graph ,G=(V,E)

The vertex V in V(G)

To visit all the vertices that are reached from

the vertex V, that is all the vertices are connected to
vertex V

There are two types of graph traversals

1) Depth first search

2) Breadth first search

12. Define depth first search?

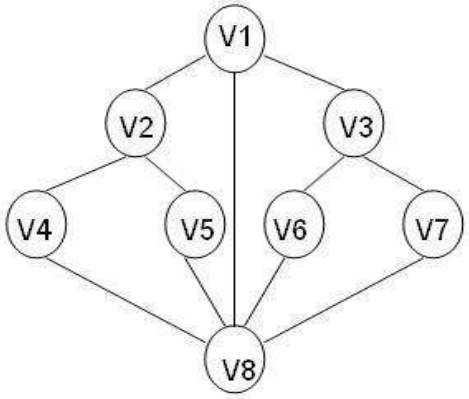
In DFS, we don't have any special vertex, we can start from any vertex.

Let us start from vertex (v), its adjacent vertex is selected and DFS is initialized.

Let us consider the adjacent vertices to V are V1, V2, and V3 …Vk.

Now pick V1 and visit its adjacent vertices then pick V2 and visit the adjacent vertices .to continue and process till the nodes are visited.

Consider the graph



V1	V2, V3, V8
V2	V4, V5
V4	V2, V8
V8	V4, V5, V1, V6, V7
V5	V2, V8
V6	V3, V8
V3	V6, V7, V1
V7	V8, V3

13. Define breadth first search?

Is BFS ,an adjacent vertex is selected then visit its adjacent vertices then backtrack the unvisited adjacent vertex

In BFS ,to visit all the vertices of the start vertex ,then visit the unvisited vertices to those adjacent vertices

14. Define topological sort?

A directed graph *G* in which the vertices represent tasks or activities and the edges represent activities to move from one event to another then the task is known as *activity on vertex network* or AOV-network/pert network

15. Define kruskal?

Kruskal's method is an approach to determine a minimum cost spanning of a graph. In this method we need to select (n-1) edges in G, edge by edge, such that these form an MST or G. An edge is included in tree if it does not form a cycle with edges already in T.

16. What is shortest path?

A path having the minimum weight between the vertices. And the length of a path is the sum of weight of the edges on that path.

17. What are the problems occur in shortest path?

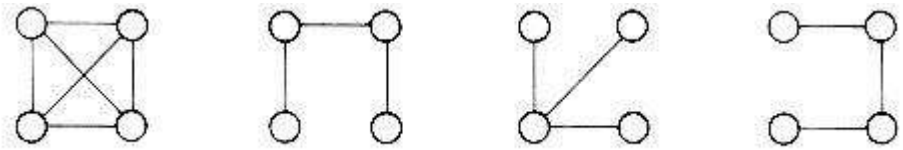
Two kinds of problems in finding shortest path

are Single source shortest path

All pair shortest path

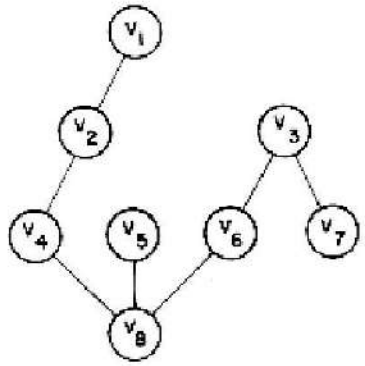
18. What is a spanning tree? (Nov 11, Nov 12, Nov 13, Nov 14, Apr 15)

A spanning tree is any tree that consists only of edges in a graph G and that includes all the vertices in G.



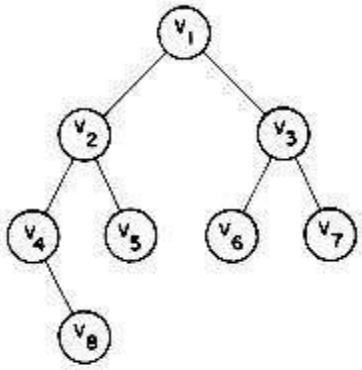
19. What DFS spanning tree?

When either DFS or BFS are used the edges of *T* form a spanning tree. The spanning tree resulting from a call to DFS is known as a *depth first spanning tree*.



20. What BFS spanning tree?

When either DFS or BFS are used the edges of *T* form a spanning tree. The spanning tree resulting from a call to BFS is called a *breadth first spanning tree.*



21. What does source and closure mean in shortest paths?

The starting vertex of the path will be referred to as the *source* and the last vertex the *destination.*

22. Differentiate depth first traversal and Breadth first traversal

Both are graph traversal algorithms. They differ in the order they reach the nodes. Depth first explores all of the nodes reachable from X before moving on to any of X's siblings, whereas breadth first search explores siblings before children. For a depth first use a stack implementation and for breadth first queue can be used.

23. Define set.

A set is an abstract data structure that can store certain values, without any particular order, and no repeated values. It is a computer implementation of the mathematical concept of a finite set. Unlike most other collection types, rather than retrieving a specific element from a set, one typically tests a value for membership in a set.

24. What are the two ways to maintain graph G in a memory of the computer? (Nov 14)

- Sequential representation of a graph using adjacent matrix
- Linked representation of a graph using linked list

11 MARKS

1. What is a graph? Write short notes on its basic terminologies.

DEFINING GRAPH

- A graphs G consists of a set V of vertices (nodes) and a set E of edges (arcs).
- We write G=(V,E). V is a finite and non-empty set of vertices.
- E is a set of pair of vertices; these pairs are called as edges .
- Therefore (G).read as V of G, is a set of vertices and E(G),read as E of G is a set of edges.
- An edge e= (v, w) is a pair of vertices v and w, and to be incident with v and w.
- **A graph can be pictorially represented as follows,**

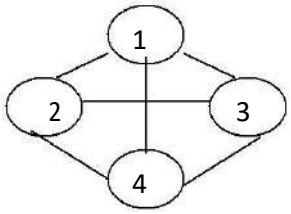


FIG: Graph G

- ✓ We have numbered the graph as 1,2,3,4.
- ✓ Therefore, V(G)=(1,2,3,4) and E(G) = {(1,2),(1,3),(1,4),(2,3),(2,4)}.

BASIC TERMINOLGIES OF GRAPH

UNDIRECTED GRAPH

An undirected graph is that in which, the pair of vertices representing the edges is unordered.

DIRECTED GRAPH

- ✓ ***A directed graph is that in which, each edge is an ordered pair of vertices, (i.e.) each edge is represented by a directed pair.***
- ✓ ***It is also referred to as digraph.***

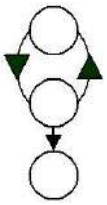


Fig.: Directed Graph

COMPLETE GRAPH

A vertex undirected graph with exactly n(n-1)/2 edges is said to be complete graph.

SUBGRAPH

A sub-graph of G is a graph G‘ such that V(G’) V(G) and E(G ‘) E(G). Some of the sub-graphs are as follow,

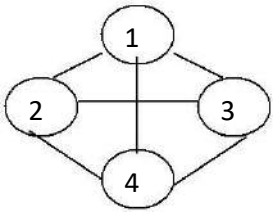
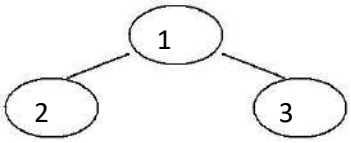
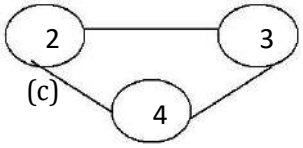


FIG: Graph G

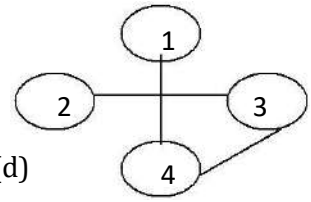
(a)



(b)



(c)



(d)

FIG: Sub graphs of G

ADJACENT VERTICES

A vertex v1 is said to be a adjacent vertex if there exist an edge (v1,v2) or (v2,v1).

PATH:

A path from vertex V to the vertex W is a sequence of vertices, each adjacent to the next. The length of the graph is the number of edges in it.

CONNECTED GRAPH

A graph is said to be connected if there exist a path from any vertex to another vertex.

UNCONNECTED GRAPH

A graph is said to be an unconnected graph if there exist any two unconnected components.

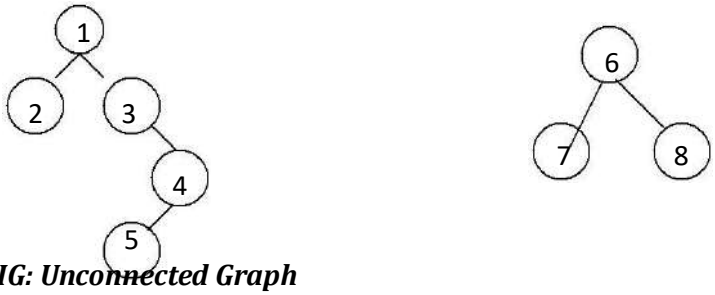
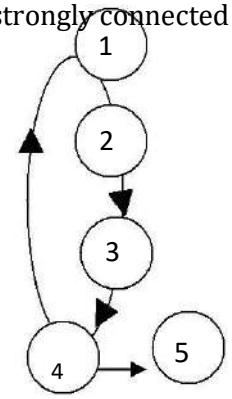


FIG: Unconnected Graph

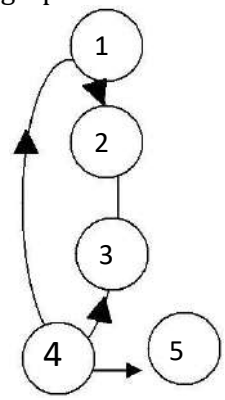
STRONGLY CONNECTED GRAPH

A digraph is said to be strongly connected if there is a directed path from any vertex to any other vertex.



WEAKLY CONNECTED GRAPH

If there does not exist a directed path from one vertex to another vertex then it is said to be a weakly connected graph.



CYCLE

A cycle is a path in which the first and the last vertices are the same.

Eg. 1,3,4,7,2,1

DEGREE:

The number of edges incident on a vertex determines its degree. There are two types of degrees In-degree and out-degree.

- IN-DEGREE of the vertex V is the number of edges for which vertex V is a head.
- OUT-DEGREE is the number of edges for which vertex is a tail.

A GRAPH IS SAID TO BE A TREE, IF IT SATISFIES THE TWO PROPERTIES:

1. It is connected
2. There are no cycles in the graph.

GRAPH REPRESENTATION:

The graphs can be represented by the follow three methods,

1. Adjacency matrix.

2. Adjacency list.
3. Adjacency multi-list.

ADJACENCY MATRIX:

The adjacency matrix A for a graph G = (V,E) with n vertices, is an n* n matrix of bits ,such that A

I_j = 1 , if there is an edge from vi to vj and

A_{ij} = 0, if there is no such edge.

The adjacency matrix for the graph G is,

$$\begin{pmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{pmatrix}$$

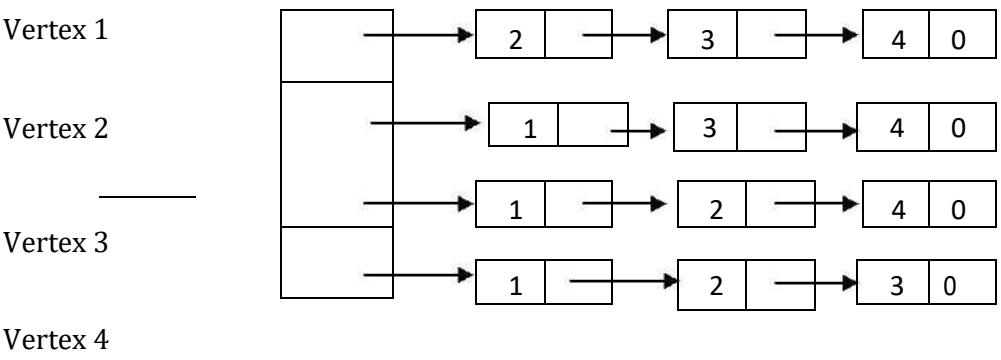
The space required to represent a graph using its adjacency matrix is n* n bits. About half this space can be saved in case of an undirected graph, by storing only the upper or lower triangle of the matrix. From the adjacency matrix, one may readily determine if there an edge connecting any two vertices I and j. for an undirected graph the degree of any vertices I is its row $\sum^{n}_{(j=1)} A(I, j)$. For a directed graph the row is the out-degree and column sum is the in-degree.

The adjacency matrix is a simple way to represent a graph , but it has 2 disadvantages,

- It takes O(n*n) space to represent a graph with n vertices ; even for sparse graphs and
- It takes… (n*n) times to solve most of the graph problems.

ADJACENCY LIST

In the representation of the n rows of the adjacency matrix are represented as n linked lists. There is one list for each vertex in G. the nodes in list I represent the vertices that are adjacent from vertex i. Each node has at least 2 fields: VERTEX and LINK. The VERTEX fields contain the indices of the vertices adjacent to vertex i. In the case of an undirected graph with n vertices and E edges, this representation requires n head nodes and 2e list nodes.

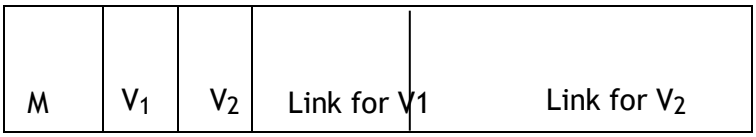


The degree of the vertex in an undirected graph may be determined by just counting the number of nodes in its adjacency list. The total number of edges in G may , therefore be determined in time O(n+e).in the case of digraph the number of list nodes is only e. the out-degree of any vertex can be determined by counting the number of nodes in an adjacency list. The total number of edges in G can, therefore be determined in O(n+e).

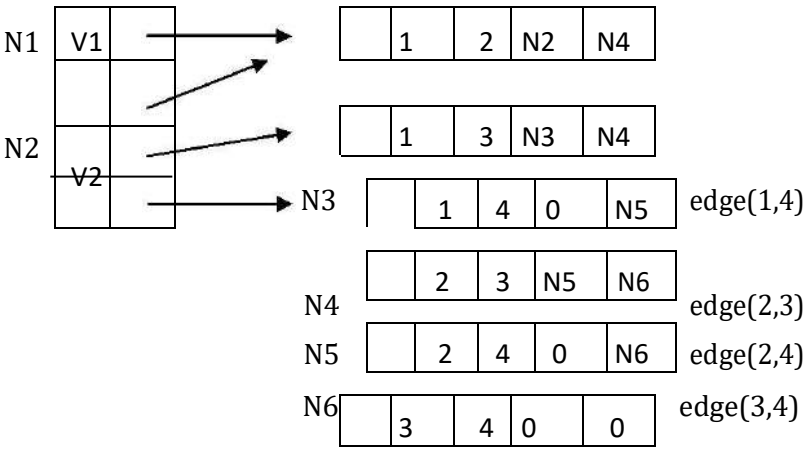
ADJACENCY MULTILIST:

In the adjacency list representation of an undirected graph each edge (v_i, v_j) is represented by two entries ,one on the list for V_i and the other on the list for V_j.

For each edge there will be exactly one node , but this node will be in two list , (i.e) the adjacency list for each of the two nodes it is incident to .the node structure now becomes



Where M is a one bit mark field that may be used to indicate whether or not the edge has been examined. The adjacency multi-list diagram is as follow,



The lists are : v1: N1 N2 N3

V2: N1 N4 → N5 →

V3: N2 N4 → N6 →

V4: N3 → N5 → N6

FIG: Adjacency Multilists for G

2. What is graph traversal? What are its types? Explain (Nov 13, Nov 14, May 15)

GRAPH TRAVERSAL

Given an undirected graph G = (V.E) and a vertex v in V(G) we are interested in visiting all vertices in G that are reachable from v (that is all vertices connected to v). We have two ways to do the traversal. They are

DEPTH FIRST SEARCH

BREADTH FIRST SEARCH.

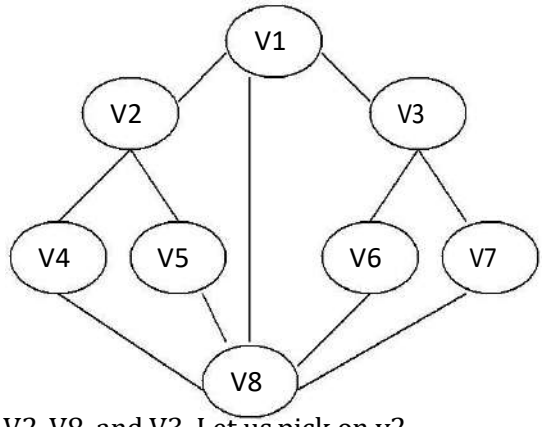
DEPTH FIRST SEARCH

In graphs, we do not have any start vertex or any special vertex singled out to start traversal from. Therefore the traversal may start from any arbitrary vertex.

We start with say, vertex v. An adjacent vertex is selected and a Depth First search is intimated from it, i.e. let V_1, V_2.. V_k are adjacent vertices to vertex v. We may select any vertex from this list. Say, we select V_1. Now all the adjacent vertices to v_1 are identified and all of those are visited; next V_2 is selected and all its adjacent vertices visited and so on.

This process continues till are the vertices are visited. It is very much possible that we reach a traversed vertex second time. Therefore we have to set a flag somewhere to check if the vertex is already visited.

Let us see an example, consider the following graph.



Let us start with V_1,

1. Its adjacent vertices are V2, V8, and V3. Let us pick on v2.
2. Its adjacent vertices are V1, V4, V5, V1 is already visited.
3. Let us pick on V4.
4. Its adjacent vertices are V2, V8.
5. V2 is already visited .let us visit V8.
6. Its adjacent vertices are V4, V5, V1, V6, V7.
7. V4 and V1 are visited. Let us traverse V5.
8. Its adjacent vertices are V2, V8. Both are already visited therefore, we back track.
9. We had V6 and V7 unvisited in the list of V8. We may visit any. We may visit any. We visit V6.
10. Its adjacent is V8 and V3. Obviously the choice is V3.
11. Its adjacent vertices are V1, V7. We visit V7.
12. All the adjacent vertices of V7 are already visited, we back track and find that we have visited all the vertices.

Therefore the sequence of traversal is V1,

V2, V4, V5, V6, V3, V7.

This is not a unique or the only sequence possible using this traversal method.

We may implement the Depth First search by using a stack, pushing all unvisited vertices to the one just visited and popping the stack to find the next vertex to visit.

This procedure is best described recursively as

in, Procedure DFS(v)

// Given an undirected graph G = (V.E) with n vertices and an array visited (n) initially set to zero . This algorithm visits all vertices reachable from v .G and VISITED are global > //VISITED (v) ← *1*

for each vertex w adjacent to v do

if VISITED (w) =0 then call DFS (w)

end

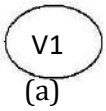
end DFS

COMPUTING TIME

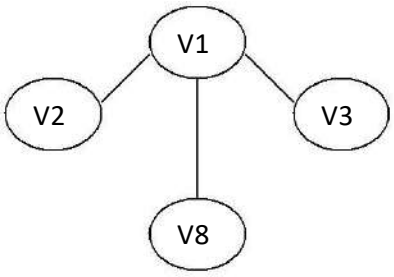
1. In case G is represented by adjacency lists then the vertices w adjacent to v can be determined by following a chain of links. Since the algorithm DFS would examine each node in the adjacency lists at most once and there are 2e list nodes. The time to complete the search is O (e).
2. If G is represented by its adjacency matrix, then the time to determine all vertices adjacent to v is O(n). Since at most n vertices are visited. The total time is $O(n^2)$.

BREADTH FIRST SEARCH

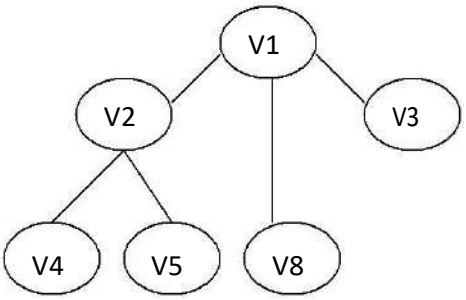
- In DFS we pick on one of the adjacent vertices; visit all of its adjacent vertices and back track to visit the unvisited adjacent vertices.
- In BFS , we first visit all the adjacent vertices of the start vertex and then visit all the unvisited vertices adjacent to these and so on.
- Let us consider the same example, given in figure. We start say, with V_1. Its adjacent vertices are V_2, V_8 , V_3.
- We visit all one by one. We pick on one of these, say V_2. The unvisited adjacent vertices to V_2 are V_4, V_5 . we visit both.
- We go back to the remaining visited vertices of V_1 and pick on one of this, say V_3. T The unvisited adjacent vertices to V_3 are V_6,V_7. There are no more unvisited adjacent vertices of V_8, V_4, V_5, V_6 and V_7.



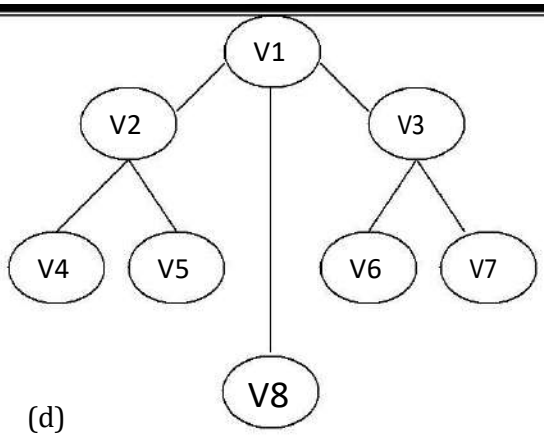
(a)



(b)



(c)



(d)

FIG: Breadth First Search

- Thus the sequence so generated is $V_1, V_2, V_8, V_3, V_4, V_5, V_6, V_7$. Here we need a queue instead of a stack to implement it.
- We add unvisited vertices adjacent to the one just visited at the rear and read at from to find the next vertex to visit.

Algorithm BFS gives the details.

Procedure BFS(v)

//A breadth first search of G is carried out beginning at vertex v. All vertices visited are marked as VISITED(I) = 1. The graph G and array VISITED are global and VISITED is initialised to 0.//

1. VISITED(v) ← 1
2. *Initialise Q to be empty //Q is a queue//*
3. loop
4. for all vertices w adjacent to v do
5. if VISITED(w) = 0 //add w to queue//
6. then [call ADDQ(w, Q); VISITED(w) ← 1] //mark w as VISITED//
7. end
8. if Q is empty then return
9. call DELETEQ(v,Q)
10. forever
11. end BFS

COMPUTING TIME

1. Each vertex visited gets into the queue exactly once, so the loop forever is iterated at most n times.
2. If an adjacency matrix is used, then the for loop takes O(n) time for each vertex visited. The total time is, therefore, $O(n^2)$.

3. In case adjacency lists are used the for loop as a total cost of $d_1+\ldots\ldots+d_n = O(e)$ where d_i = degree(v_i). Again, all vertices visited. Together with all edges incident to from a connected component of G.

3. Explain about spanning trees in detail.

SPANNING TREE:

When the graph G is connected, a depth first search starting at any vertex, visits all the vertices in G.

In this case the edges of G are partitioned into two sets T (for tree edges) and B (for back edges), where T is the set of edges used or traversed during the search and B the set of remaining edges.

The set T may be determined by inserting the statement

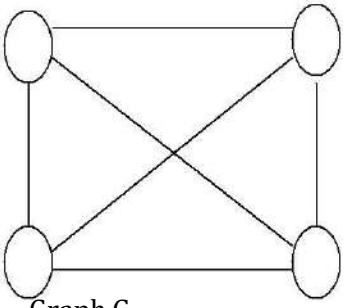
$T \leftarrow T \cup \{(v,w)\}$

In the then clauses of DFS and BFS.

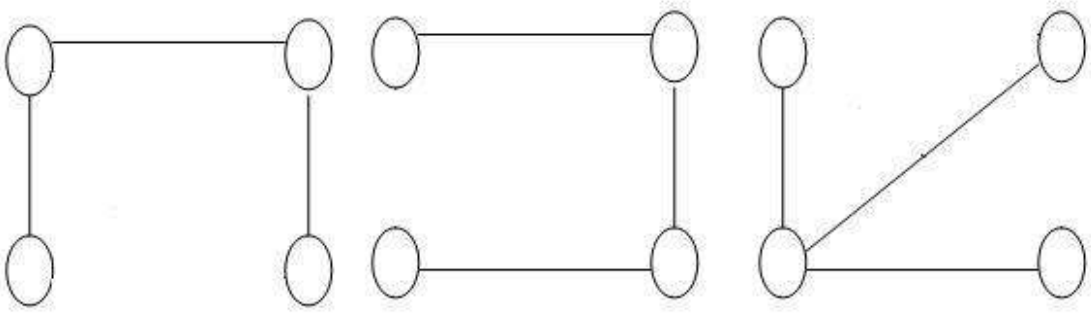
The edges in T form a tree which includes all the vertices of G

Any tree consisting solely of edges in G and including all vertices in G is called **spanning tree.**

The below diagram shows the graph G and some of its spanning trees.



Graph G



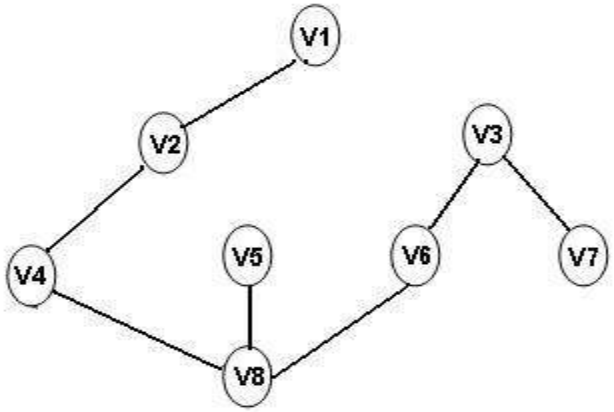
Its spanning trees

DEPTH FIRST SPANNING TREE :

When DFS are used the edges of T form a spanning tree

The spanning tree resulting from a call to DFS is known as a depth first spanning tree.

The below diagram shows the Depth first spanning tree



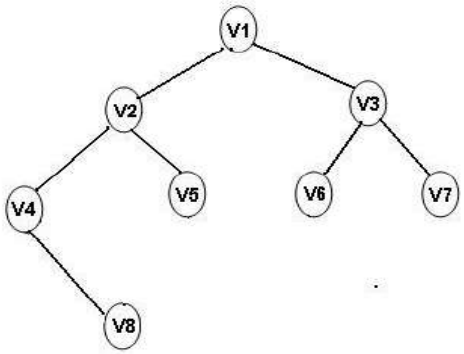
BREADTH FIRST SPANNING TREE :

When BFS are used the edges of T form a spanning tree.

The spanning tree resulting from a call to BFS is known as a breadth first spanning tree

.

The below diagram shows the breadth first spanning tree



SPANNING TREE

A tree is a spanning tree of a connected graph G (V, E) such that,

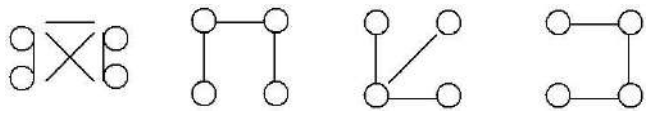
Every vertex of G belongs to an edge in T and

The edges in T form a tree.

CONSTRUCTING A SPANNING TREE

- Let us see how we construct a spanning tree for a given graph.

- Take any vertex v as an initial partial tree and edges one by one so that each edge gives a new vertex to the partial tree.
- In general if there is a vertex in the graph we shall construct a spanning tree in (n-1) steps i.e. (n-1) edges are needed to be added.
- The following figure shows a complete graph and three of its spanning trees.

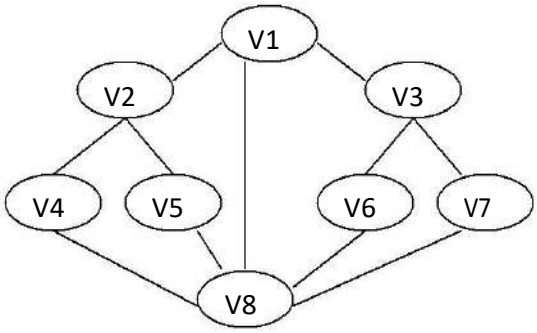


When either DFS or BFS are used the edges of T form a spanning tree.

The spanning tree resulting from a call to DFS is known as a ***depth first spanning tree***.

When BFS is used the resulting spanning tree is called a ***breadth first spanning tree***.

Consider the graph,



The following figure shows the spanning trees resulting from depth first add breadth first search starting at vertex V1 in the above graph.

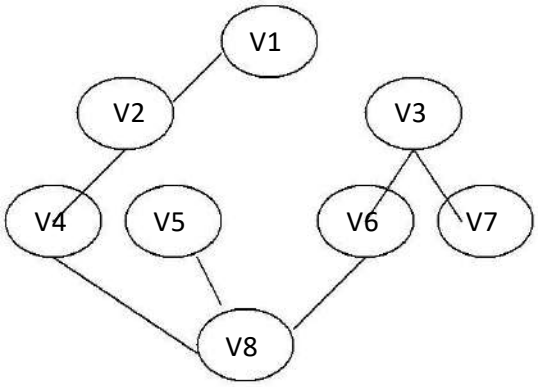


Fig.: DFS (I) spanning tree.

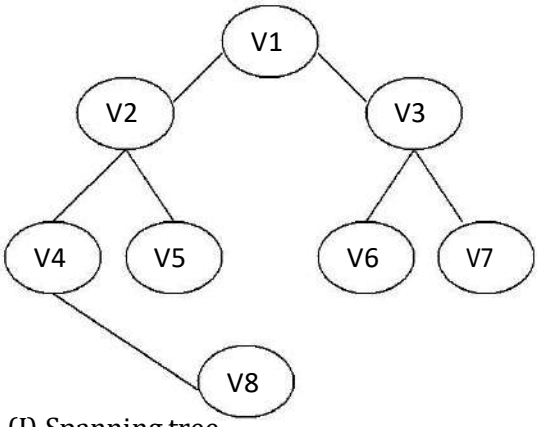


Fig.: BFS (I) Spanning tree

MINIMUM COST SPANNING TREES (APPLICATIONS OF SPANNING TREE):

BUILDING A MINIMUM SPANNING TREE

- If the nodes of G represent cities and the edges represent possible communication links connecting two cities then the minimum number of links needed to connect the n cities is n-1. The spanning tree of G will represent all feasible choices.

- This application of spanning tree arises from the property that a spanning tree is a minimal sub-graph G' of G such that V (G') =V (G) and G' is connected where minimal sub-graph is one with fewest number of edges.

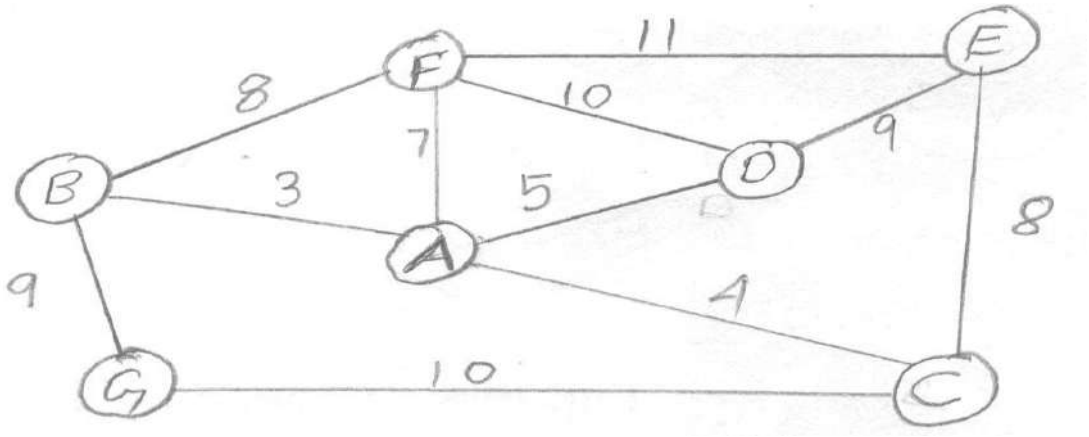
- The edges will have weights associated to there i.e. cost of communication. Given the weighted graph, one has to construct a communication links that would connect all the cities with minimum total cost.

- Since, the links selected will form a tree. We are going to find the spanning tree of a given graph with minimum cost. The cost of a spanning tree is the sum of the costs of the edges in the tree.

4. Explain about kruskal's method for creating a spanning tree (Nov 14)

One approach to determine a minimum cost spanning of a graph has been given by kruskal. In this approach we need to select (n-1) edges in G, edge by edge, such that these form an MST or G. We can select another least cost edge and so on. An edge is included in the tree if it does not form a cycle with the edges already in T.

For example, consider the following graph,



The minimum cost edge is that connects vertex A and B. let us choose that Fig.(a)

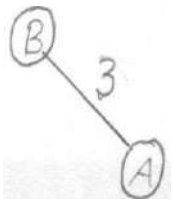


Fig. (a)

Now we have, Vertices that may be added,

	Edge	cost
F	BF, AF	8, 7
G	BG, CG	9, 10
D	AD	5
E	CE	8
C	AC	4

The least cost edge is AC therefore we choose AC. Now we have Fig. (b),

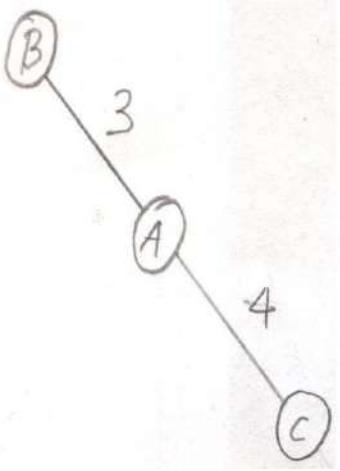
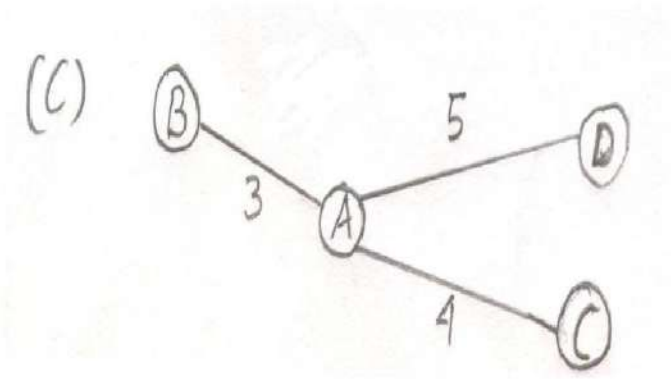


Fig. b

Vertices that may be added,

	EDGE	COST
F	BF	8
	AF	7
G	BG	9
	CG	10
D	AD	5
E	CE	8
C	AC	4

The least cost edge is Ad therefore we have



Vertices that may be added,

	EDGE	COST
F	BF	8
	AF	7
	DF	10
G	BG	9
	CG	10
E	CE	8
	DE	9

AF is the minimum cost edge therefore we add it to the partial tree we have

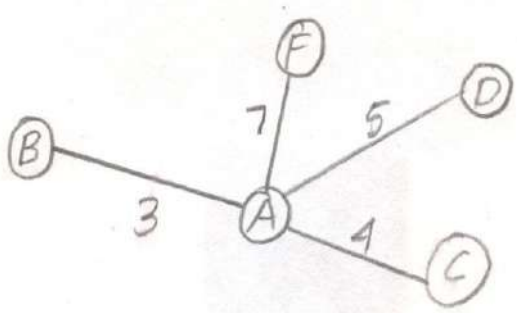


Fig.: (d)

Vertices that may be added,

	edge	cost
G	BG	9
	CG	10
E	CE	8
	DE	9
	FE	11

Obvious choice is CE, shown in figure (e),

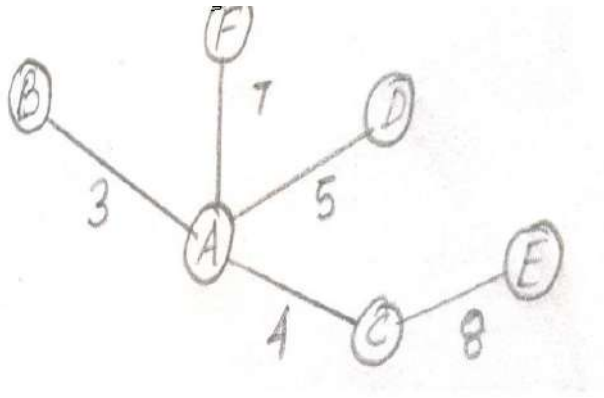
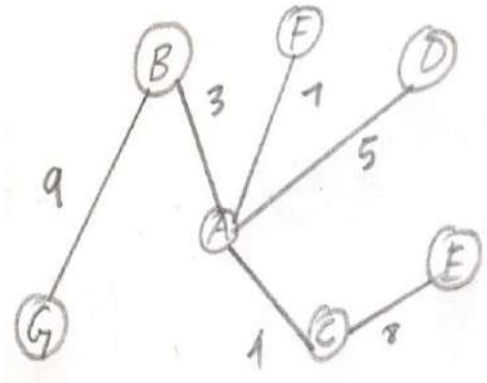


Fig. (e)

The only vertex left is G and we have the minimum cost edge that connects it to the tree constructed so for is BG. Therefore add it and the costs 9+3+7+5+4+8+36 and is given by the following figure,



ALGORITHM

1. T<-0
2. While T contains less than(n-1) edges and E not empty do
3. Choose an edge(V,W) from E of lowest cost
4. Delete (V,W) from E
5. if (V,W) does not create a cycle in T
6. then add(V,W) to T
7. else discard (V,W)

8. end
9. if T contains fewer than (n-1) edges then print (no spanning tree)

Initially, E is the set of all edges in G. In this set, we are going to (I) determining an edge with minimum cost (in line 3)

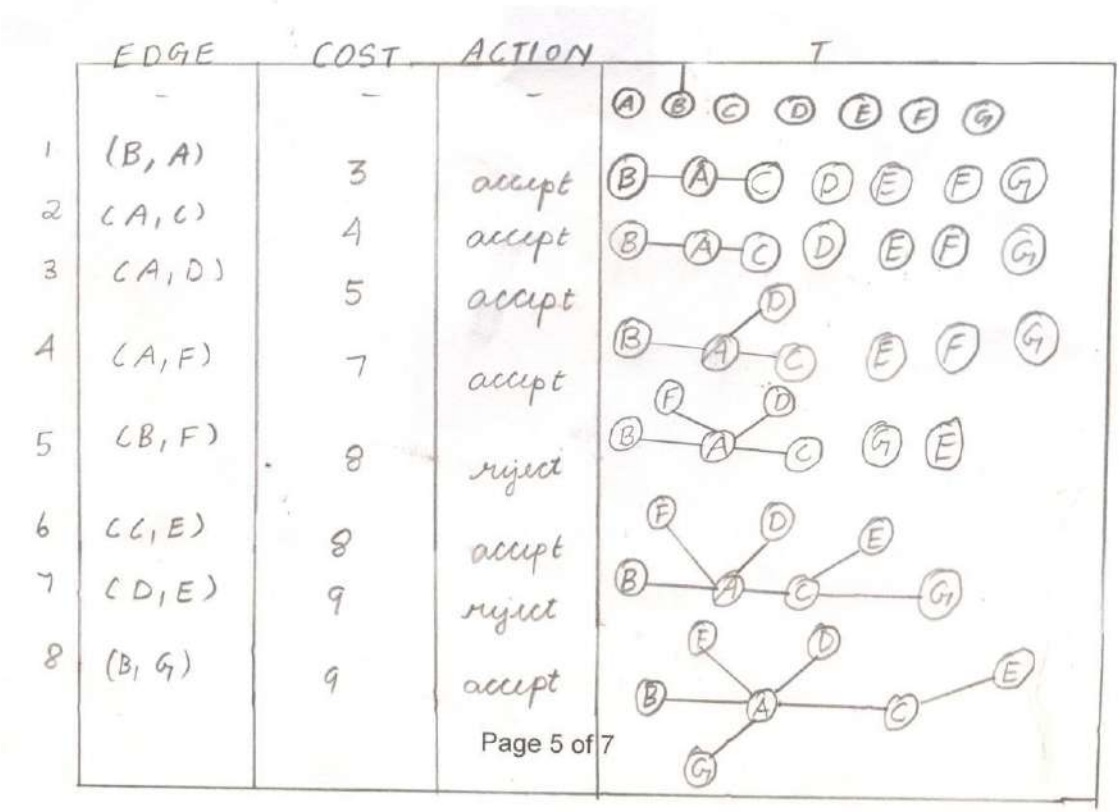
(ii) Delete that edge (line 4)

This can be done efficiently if the edges in E are maintained as a sorted sequential list.

In order to be able to perform steps 5&6 efficiently, the vertices in G should be grouped together in such way that one may easily determine of the vertices V and W are already connected by the earlier selection of edges.

In case they are, then the edge (V,W) is to be discarded. If they are not then (V,W) is to be added to T.

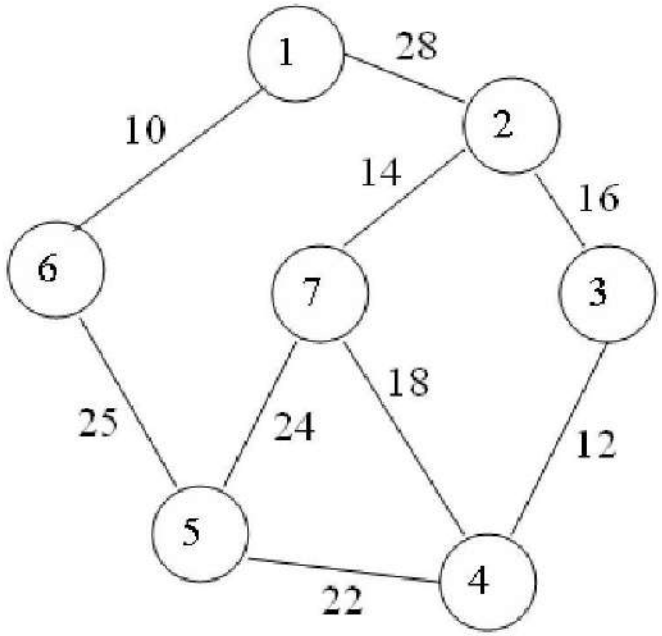
Our possible grouping is to place all vertices in the same connected components of T into the set. Then two vertices V,W are connected in t if they are in the same set.



Here for example when the edge (B,F) is to be considered, the sets would be {A,B,C,D,F},{E},{F}.vertices B and F are in the same set and so the edge (B,F) is rejected. The next edge to be considered is (C, E).since vertices C and E are in different sets the edge is accepted. This edge connects the two components {A, B, C, D, F} and {E} together and so these two sets should be joined to obtain the set representing the new component.

5. Explain about PRIM'S ALGORITHM in detail (Nov 14)

Start from an arbitrary vertex (root). At each stage, add a new branch (edge) to the tree already constructed; the algorithm halts when all the vertices in the graph have been reached.



Algorithm prims(e,cost,n,t)

{

Let (k,*l*) be an edge of minimum cost in E;

Mincost :=cost[k,*l*];

T[1,1]:=k; t[1,2]:=*l*;

For I:=1 to n do

If (cost[i,*l*]<cost[i,k]) then near[i]:=*l*;

Else near[i]:=k; Near[k]:=near[l]:=0;

For i:=2 to n-1 do

{

Let j be an index such that near[j]≠0 and

Cost[j,near[j]] is minimum;

T[i,1]:=j; t[i,2]:=near[j];

Mincost:=mincost+ Cost[j,near[j]];

Near[j]:=0;

For k:=0 to n do

If near((near[k]≠0) and (Cost[k,near[k]]>cost[k,j])) then

Near[k]:=j;

}

Return mincost;

}

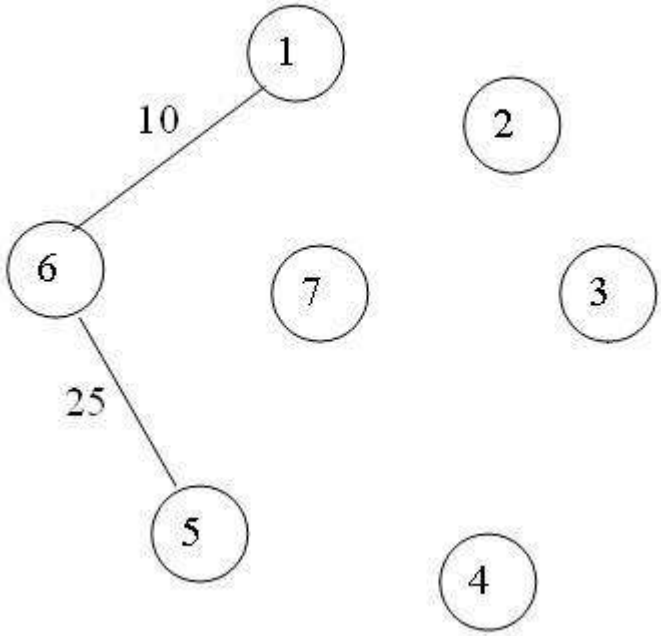
The prims algorithm will start with a tree that includes only a minimum cost edge of G.

Then, edges are added to the tree one by one. the next edge (i,j) to be added in such that I is a vertex

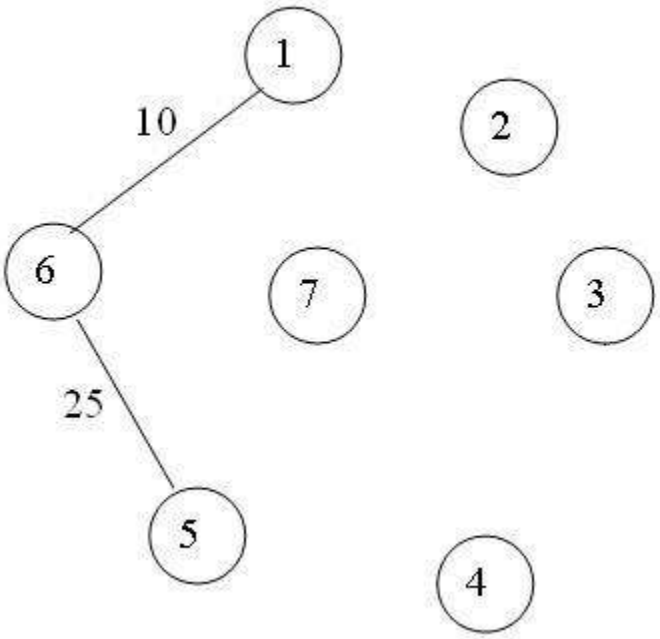
included in the tree, j is a vertex not yet included, and cost of (i,j), cost[i,j] is minimum among all the edges.

The working of prims will be explained by following diagram

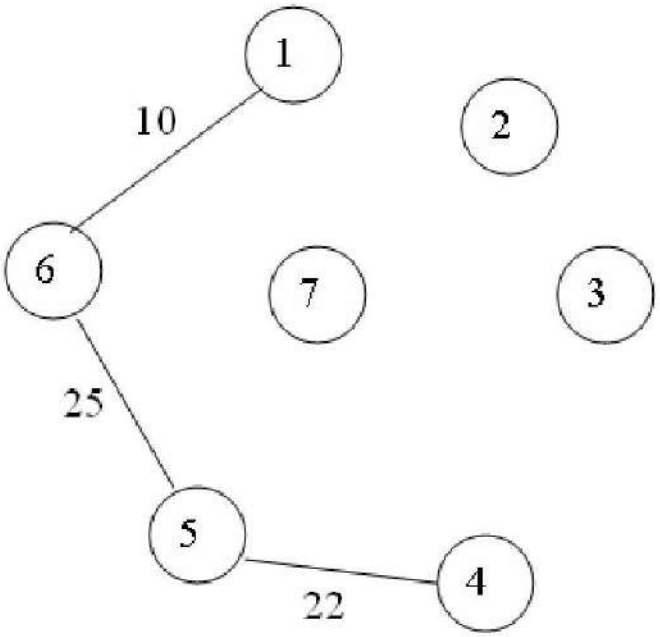
Step 1:

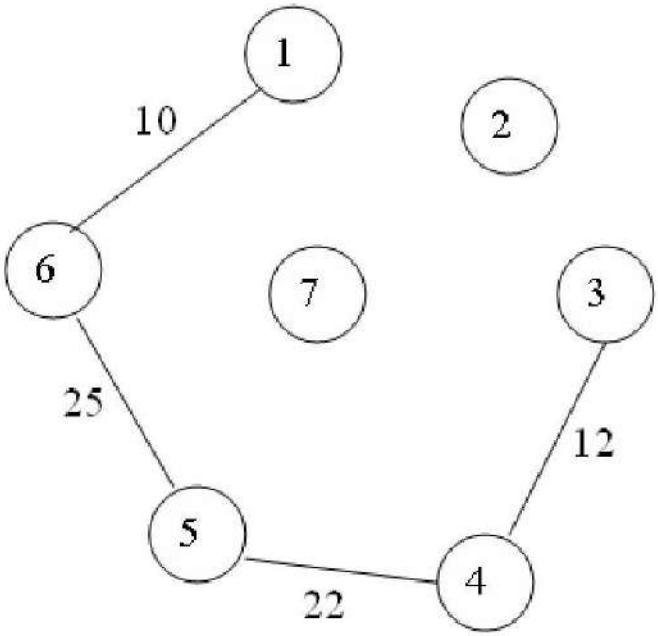


step:2

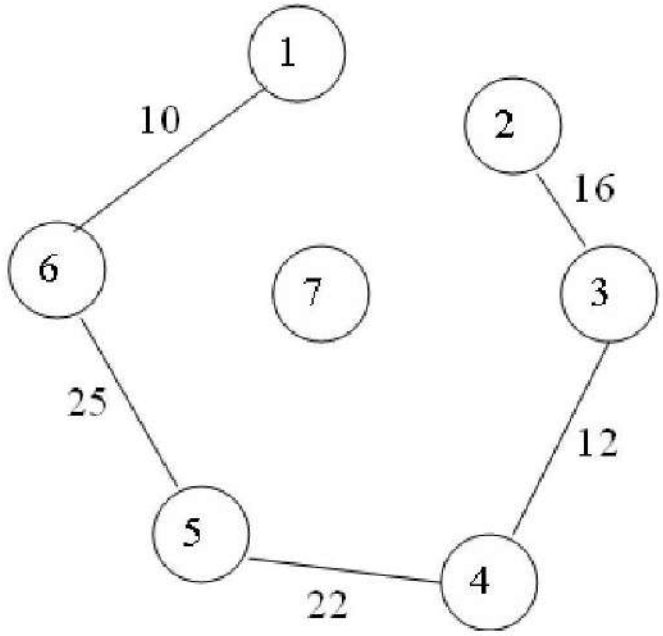


Step3:



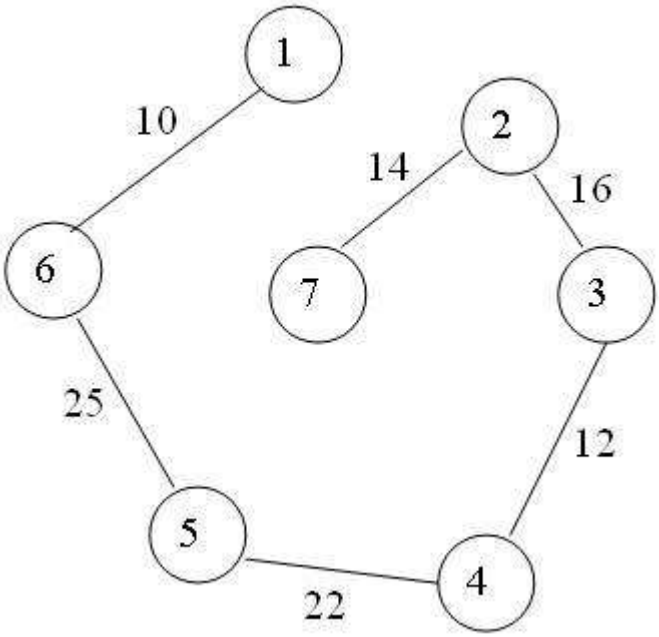


Step 4:



Step 5:

Step 6:



6.Explain about Dijikstra's algorithm: (Apr 12, Nov 12, Nov 13, Nov 14)

Procedure Dijikstra

{ Dijikstra computes the cost of the shortest path from vertex 1 to every vertex of a directed graph
} begin

1. s={1};
2. for i:=2 to n do
3. D[i]:=c[1,i];{initialize D}
4. for i=1 to n-1 do begin

5. choose a vertex w in V-S such that D[w] is minimum

6. Add w to S

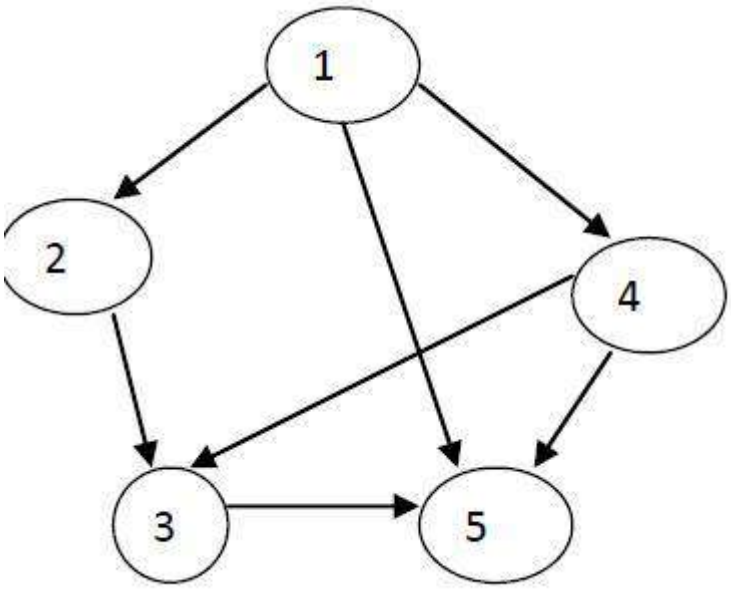
7. for each vertex v in V-S do

8. D[V]:=min(D[v],D[w]+c[w,v]);

9. end

end{Dijikstra}

Example 2:



Cost adjacency matrix:

1 2 3 4 5

1 9999 10 9999 30 100

2 9999 9999 50 9999 9999

3 9999 9999 9999 9999 10

4 9999 9999 20 9999 60

5 9999 9999 9999 9999

9999 S={1}

V={1,2,3,4,5}

For i=2 to 5 do

i=2 D[2]=10

i=3 D[3]=9999

i=4 D[4]=30

i=5 D[5]=100

for i=1 to 4 do

D[3]=60

D[5]=30

i=1 V-

S={2,3,4,5}

D[w]=10;

w=2(minimum)

S={1,2}

For each vertex v in V-S i.e)V-S={3,4,5}=>for v=3,4,5

When v=3

D[3]=min(D[3],D[2]+c[2,3])

D[3]=min(9999,10+50)

D[3]=60

1

2

4

3 5

When v=4

D[4]=min(D[4],D[2]+c[2,4])

D[4]=min(30,10+9999)

D[4]=30

When v=5

D[5]=min(D[5],D[2]+c[2,5])

D[5]=min(100,10+9999)

D[5]=100

D[2]=10 D[3]=60 D[4]=30 D[5]=100

i=2

V-S={3,4,5}

D[w]=30; w=4(minimum)

S={1,2,4}

For each vertex v in V-S{3,5}=>for v=3,5

When v=3

D[3]=min(D[3],D[4]+c[4,3])

D[3]=min(60,30+20)

D[3]=50

When v=5

D[5]=min(D[5],D[4]+c[4,5])

D[5]=min(100,30+60)

D[5]=90

D[3]=50

D[5]=90 i=3

V-S={3,5}

D[w]=50; w=3(minimum)

S={1,2,4,3}

For each vertex v in V-S={5}=>for

v=5 When v=5

D[5]=min(D[5],D[3]+c[3,5])

D[5]=min(90,50+10)

D[5]=60

D[5]=60

i=4 V-

S={5}

D[w]=60; w=5(minimum)

S={1,2,4,3,5}

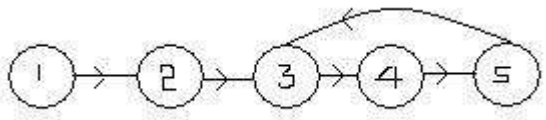
Analysis:

Run time is 0((n + |E|) log n).

7. Explain transitive closure with examples.

A problem related to the all pairs shortest path problem is that of determining for every pair of vertices *i,j* in *G* the existence of a path from *i* to *j*. Two cases are of interest, one when all path lengths (i.e., the number of edges on the path) are required to be positive and the other when path lengths are to be nonnegative. If *A* is the adjacency matrix of *G*, then the matrix A^+ having the property A^+ (*i,j*) = 1 if there is a path of length > 0 from *i* to *j* and 0 otherwise is called the *transitive closure* matrix of *G*. The matrix A^* with the property A^* (*i,j*) = 1 if

there is a path of length 0 from *i* to *j* and 0 otherwise is the *reflexive transitive closure* matrix of *G*.



a Digraph G

$$\begin{bmatrix} 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 & 0 \end{bmatrix}$$

b Adjacency matrix A for G

$$\begin{bmatrix} 1 & 1 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 & 1 \end{bmatrix}$$

c A^+

$$\begin{bmatrix} 1 & 1 & 1 & 1 & 1 \\ 0 & 1 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 & 1 \end{bmatrix}$$

d A^*

Figure Graph G and Its Adjacency Matrix A, A^+ and A^*

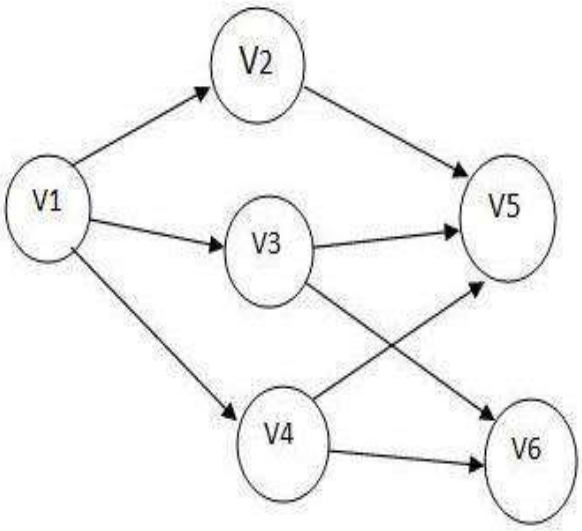
Figure shows A^+ and A^* for a digraph. Clearly, the only difference between A^* and A^+ is in the terms on the diagonal. $A^+(i,i) = 1$ iff there a cycle of length > 1 containing vertex i while $A^*(i,i)$ is always one as there is a path of length 0 from i to i. If we use algorithm ALL_COSTS with COST(i,j) = 1 if <i,j> is an edge in G and COST(i,j) = +

if <i,j> is not in G, then we can easily obtain A^+ from the final matrix A by letting A^+ (i,j) = 1 iff A (i,j) < $+\infty$. A^* can be obtained from A^+ by setting all diagonal elements equal 1. The total time is $O(n^3)$. Some simplification is achieved by slightly modifying the algorithm. In this modification the computation of line 9 of ALL_COSTS becomes $A(i,j) \leftarrow A$ (i,j) **or** ($A(i,k)$ **and** $A(k,)$) and COST(i,j) is just the adjacency matrix of G. With this modification, A need only be a bit matrix and then the final matrix A will be A^+.

8. Explain the topological sort with an example. (Nov 11, Apr 13, Nov 14, May 14)

TOPOLOGICAL SORT

- A directed graph G in which the vertices represent event and the edges represent activities to move from one event to another then that graph is known as activity on vertex network (or) AOV network (or) PERT graph.
- PERT graph is used to analyze interrelated activities of complete projects
- The purpose of topological sort is to order the events in a linear manner with the restriction that an event cannot precede other event that must first take place
- Let us assume that we have to do a set of jobs. But certain jobs have to be performed after completion of other job.
- Topological sort defines an order to do these jobs such that each job is performed only after all of its constraints are satisfied.
- Topological sorting is an ordering of the vertices in a directed acyclic graph, such that , if there is a path from x to y, then y appears after x and not x after y in a cycle.

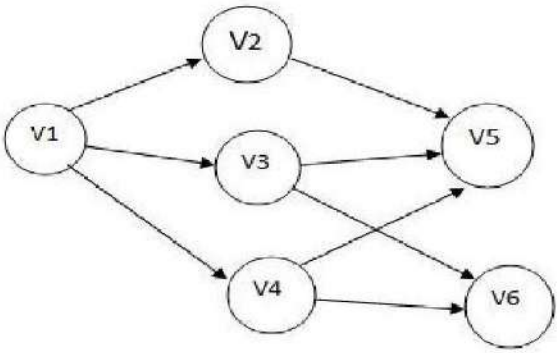


ALGORITHM FOR TOPOLOGICAL SORTING

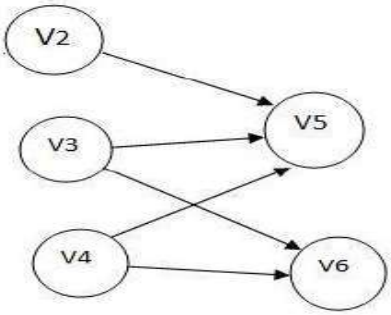
- Step 1 : For I -> 1 to n do // o/p the vertices // .
- Step 2 : If every vertex has no predecessor then (the net work has a cycle and stop).
- Step 3 : Pick a vertex which has no predecessor.
- Step 4 : Output V.
- Step 5 : Delete V and all edges leading and of V from the network.
- Step 6 : End.

Applying this for above graph, we

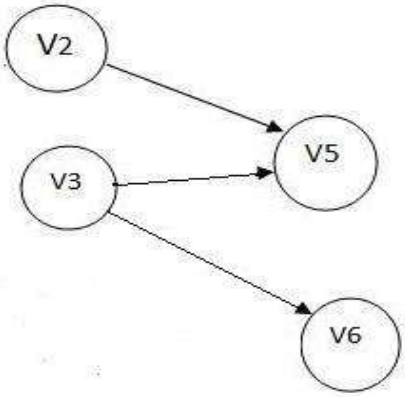
get:
- A) Initial



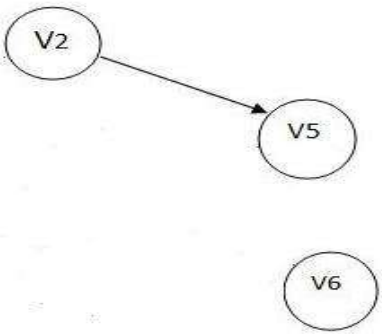
❖ B) V1



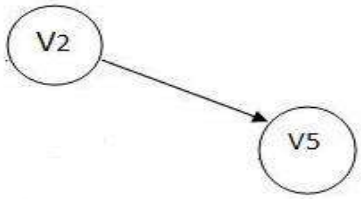
❖ C) V4



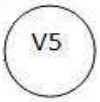
❖ D) V3



- E) V6



- F) V2



- G) V5
- Topological order generated is : V1, V4, V3, V6, V2, V5.
- In step B) we have 3 vertices V2, V3, &V4 which has no predecessors and any of these can be the next vertex in the topological order.
- In this problem, the functions are :
    1. Decide whether a vertex has any predecessors.
    2. Decide a vertex together with all its incident edges.
    3. This can be efficiently done if for each vertex a count of the number of its immediate predecessor is kept.
    4. This is easily implemented if the network is represented as a adjacency list.

The following algorithm assumes that the network is represented as adjacency list.

ALGORITHM

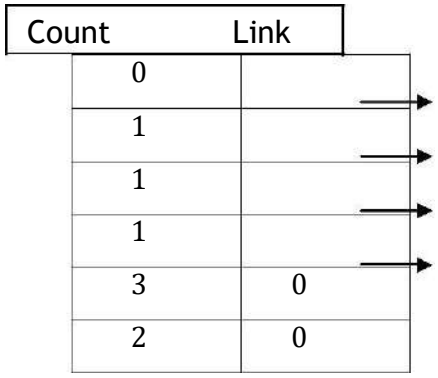
Procedure TOPOLOGICAL _ORDER (count, vertex, link, n)

//The n vertices of an AOV network are listed in topological order. The network is represented as a set of adjacency lists with

1. COUNT(I) = the in-degree of vertex I //
2. Top <- 0 //initialize stack//
3. For I <-1 to n do //create a linked stack of vertices with no predecessors//
4. If COUNT (I) = 0 then [COUNT(I) <- top; top <- I]
5. End
6. For I <-1 to n do //print the vertices in topological order//
7. If top = 0 then [print (network has a cycle) ;stop]
8. J <- top ; top <-COUNT (top) ; print(j) //unstack a vertex//
9. Ptr <-LINK (j)
10. While ptr != 0 do //decrease the count of successor vertices of j //

11. K <- VERTEX (ptr) //K is a successor of j //
12. COUNT (K) <- COUNT (k)-1 //decrease count//
13. If COUNT (K) =0 //add vertex K to stack//
14. Then [COUNT (K) <- top; top <- K]
15. Ptr <- LINK (ptr)
16. End
17. End
18. End TOPOLOGICAL _ORDER.

VERTEX LINK



The head nodes of these lists contain two fields: COUNT and LINK. The COUNT field contains the in-degree of that vertex and LINK is a pointer to the first node on the adjacency list. Each list node has two fields: VERTEX and LINK.

COUNT field can be set at the time of input. When edge (I , j) is the input the count of vertex j is incremented by 1. The list of vertices with zero count is maintained as a stack.

9. Describe the applications of graph. (Nov 11)

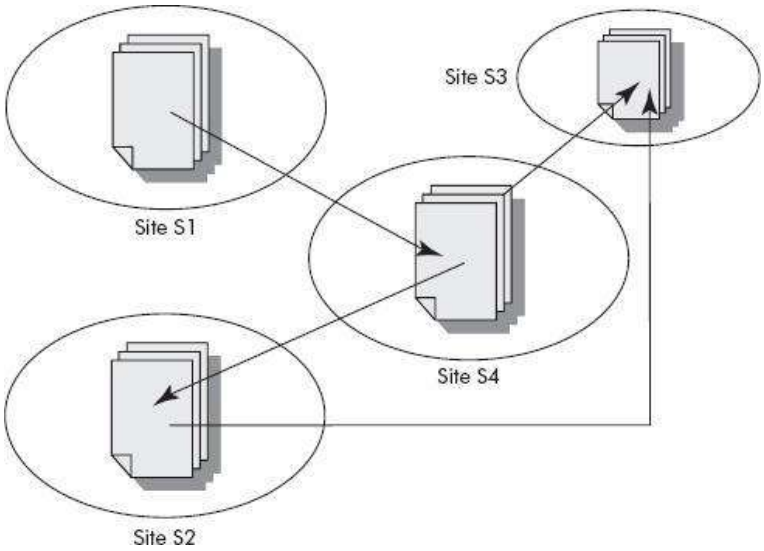
Structures that can be represented as graphs are ubiquitous, and many problems of practical interest can be represented by graphs. The link structure of a website could be represented by a directed graph: the vertices are the web pages available at the website and a directed edge from page A to page B exists if and only if A contains a link to B. A similar approach can be taken to problems in travel, biology, computer chip design, and many other fields. The development of algorithms to handle graphs is therefore of major interest in computer science. There, the transformation of graphs is often formalized and represented by graph rewrite systems. They are either directly used or properties of the rewrite systems(e.g. confluence) are studied.

A graph structure can be extended by assigning a weight to each edge of the graph. Graphs with weights, or weighted graphs, are used to represent structures in which pairwise connections have some numerical values. For example if a graph represents a road network, the weights could represent the length of each road. A digraph with weighted edges in the context of graph theory is called a network.

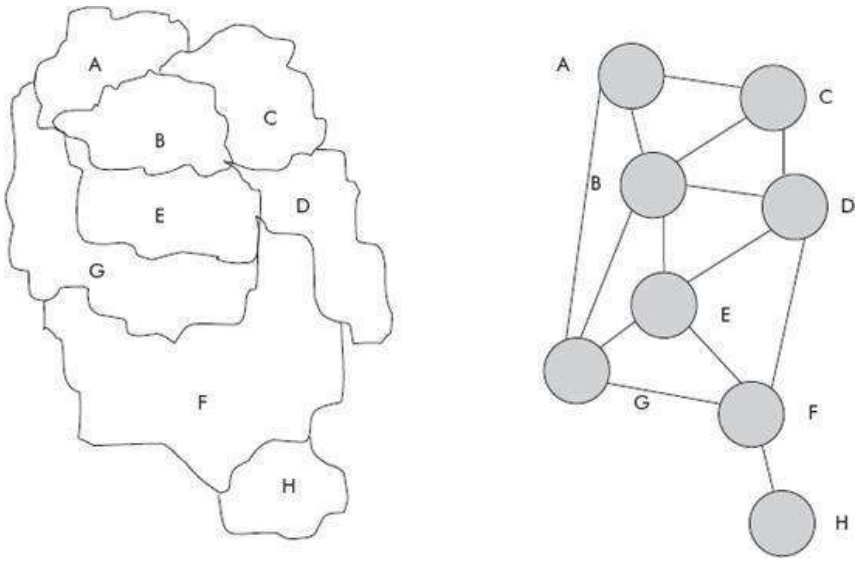
Networks have many uses in the practical side of graph theory, network analysis (for example, to model and analyze traffic networks). Within network analysis, the definition of the term "network" varies, and may often refer to a simple graph.

Model of www: The model of world wide web (www) can be represented by a graph (directed) wherein nodes denote the documents, papers, articles, etc. and the edges represent the outgoing hyperlinks between them as shown in

Figure



Coloring of maps: Coloring of maps is an interesting problem wherein it is desired that a map has to be colored in such a fashion that no two adjacent countries or regions have the same color. The constraint is to use minimum number of colors. A map can be represented as a graph wherein a node represents a region and an edge between two regions denote that the two regions are adjacent.



10. What is a set? How it is represented? Explain its operations. (Nov 14)

One of the most basic mathematical constructs is a *set*, which is an unordered collection of unique objects. The objects contained within a set are referred to as the set's *elements*.

Examples of this notation can be seen below:

S = { 1, 3, 5, 7, 9 }

T = { Scott, Jisun, Sam }

U = { -4, 3.14159, Todd, x }

The Fundamentals of Sets

The number of unique elements in a set is the set's *cardinality*. The set {1, 2, 3} has cardinality 3, just as does the set {1, 1, 1, 1, 1, 1, 1, 2, 3} (because it only has three unique elements). A set may have no elements in it at all. Such a set is called the *empty set*, and is denoted as {} or Æ, and has a cardinality of 0.

A set, on the other hand, is *unordered* and contains *unique* items. Since sets are unordered, the elements of a set may be listed in any order. That is, the sets {1, 2, 3} and {3, 1, 2} are considered equivalent. Also, any duplicates in a set are considered redundant. The set {1, 1, 1, 2, 3} and the set {1, 2, 3} are equivalent. Two sets are equivalent if they have the same elements. (Equivalence is denoted with the = sign; if S and T are equivalent they are written as $S = T$.)

Set Operations

Common operations on numbers include addition, multiplication, subtraction, exponentiation, and so on. For sets, there are four basic operations:

Union – the union of two sets, denoted S È T, is akin to addition for numbers. The union operator returns a set that contains all of the elements in S and all of the elements in T. For example, {1, 2, 3} È {2, 4, 6} equals {1, 2, 3, 2, 4, 6}. (The duplicate 2 can be removed to provide a more concise answer, yielding {1, 2, 3, 4, 6}.) Formally, S È T = {x : x Î S or x Î T}. In English, this translates to S union T results in the set that contains an element x if x is in S or in T.

Intersection – the intersection of two sets, denoted S Ç T, is the set of elements that S and T have in common. For example, {1, 2, 3} Ç {2, 4, 6} equals {2}, since that's the only element both {1, 2, 3} and {2, 4, 6} share in common. Formally, S Ç T = {x : x Î S and x Î T}. In English, this translates to S intersect T results in the set that contains an element x if x is both in S **and** in T.

Difference – the difference of two sets, denoted S - T, are all of the elements in S that are **not** in T. For example, {1, 2, 3} - {2, 4, 6} equals {1, 3}, since 1 and 3 are the elements in S that are not in T. Formally, S - T = {x : x Î S and x Ï T}. In English, this translates to S set difference T results in the set that contains an element x if x is in $S$and **not** in T.

Complement – Earlier we discussed how typically sets are limited to a known universe of possible values, such as the integers. The complement of a set, denoted *S*', is *U- S*. (Recall that *U* is the universe set.) If our universe is the integers 1 through 10, and *S* = {1, 4, 9, 10}, then *S*' = {2, 3, 5, 6, 7, 8}. (Complementing a set is akin to negating a number. Just like negating a number twice will give you the original number back—that is, --*x* = *x*—complementing a set twice will give you the original set back—*S*'' =*S*.)

PONDICHERRY UNIVERSITY QUESTIONS
11 MARKS

NOV 2011(REGULAR)

1. Explain the topological sort with an example. (Pg. No. 30) (Qn. No. 8)

2. Describe the applications of graph. (Pg. No. 33) (Qn. No. 9)

MAY 2012(ARREAR)

1.Explain about Dijikstra's algorithm (Pg. No.25) (Qn. No. 6)

NOV 2012(REGULAR)

1.Explain about Dijikstra's algorithm (Pg. No.25) (Qn. No. 6)

MAY 2013(ARREAR)

1. Explain the topological sort with an example. (Pg. No. 30) (Qn. No. 8)

NOV 2013 (REGULAR)

1.Explain about Dijikstra's algorithm: (Pg. No. 25) (Qn. No. 6)

2. What is graph traversal? What are its types? Explain (Pg. No. 10) (Qn. No. 2)

MAY 2014(ARREAR)

1. Explain the topological sort with an example. (Pg. No. 30) (Qn. No. 8)

NOV 2014 (REGULAR)

1. Write an algorithm to find the minimum cost spanning tree of an undirected graph. (Pg. No. 17,21) (Qn. No. 4,5)
2. Write short notes on breadth first search with an example. (Pg. No. 10) (Qn. No. 2)
3. Explain Kruskal's algorithm for computing the minimum spanning tree for weighted undirected graph. (Pg. No. 17) (Qn. No. 4)
4. Explain the procedure to find the shortest path for the given graph. Develop and analyze your procedure. (Pg. No. 25) (Qn. No. 6)
5. Describe the topological sort with example. (Pg. No. 30) (Qn. No. 8)
6. How can you represent set as a tree? What are the operations that we can perform on the sets? How these operations are implemented? (Pg. No. 35) (Qn. No. 10)

MAY 2015(ARREAR)

1. Explain in detail about depth first search (Pg. No. 10) (Qn. No. 2)
2. Write the routine for performing breadth first search. (Pg. No. 10) (Qn. No. 2)

NOV 2015(REGULAR)

1.Give Notes on (a.)Representation of Sets (b.)Operation on sets.

2. Define Graph.How is it represented?Explain its traversal schemes with an Example

MAY 2016 (ARREAR)

1.Explain BFS and DFS in details

2. Explain the single source shortest path algorithm with an example.